TIC-TAC-TOE GAME DEVELOPMENT USING PYTHON

AIM

To design and develop an interactive two-player Tic-Tac-Toe game using Python Tkinter. The game allows two players to take turns placing X and O symbols on a 3×3 grid, detects winning combinations and draws, displays the game result, and provides restart and exit options.

PROCEDURE

1. Start the program and import the required Python library, tkinter.
2. Create the main application window and set its title, size, and background colour.
3. Design a 3×3 game board using Tkinter buttons arranged in a grid.
4. Initialize an empty board and set Player X as the first player.
5. Allow players to select an empty cell by clicking its button.
6. Display the current player's symbol (X or O) in the selected cell.
7. After each move, check the rows, columns, and diagonals for a winning combination.
8. If a player gets three matching symbols in a row, display the winner and disable further moves.
9. If all cells are filled without a winner, display a draw message.
10. Provide a Restart Game button to clear the board and start a new game.
11. Provide an Exit button to close the application.
12. Run the program and test turns, win detection, draw detection, restart, and exit functions.

PROGRAM CODE

```
import tkinter as tk
from tkinter import messagebox

# Game Settings
BG = "#172554"
BOARD_BG = "#0F172A"
X_COLOR = "#86EFAC"
O_COLOR = "#FACC15"

class TicTacToe:
    def __init__(self, root):
        self.root = root
        self.root.title("TIC-TAC-TOE")
        self.root.resizable(False, False)
        self.root.configure(bg=BG)

        tk.Label(
            root, text="TIC-TAC-TOE",
            font=("Arial Black", 22),
            bg=BG, fg="white"
        ).pack(pady=10)

        self.status = tk.Label(
            root, text="PLAYER X'S TURN",
            font=("Arial", 14, "bold"),
            bg=BG, fg="#FACC15"
        )
        self.status.pack(pady=5)

        self.board_frame = tk.Frame(root, bg=BOARD_BG)
        self.board_frame.pack(padx=20, pady=10)

        self.buttons = []
        for row in range(3):
            button_row = []
            for col in range(3):
                button = tk.Button(
                    self.board_frame, text="",
                    font=("Arial Black", 28),
                    width=4, height=2,
                    bg=BOARD_BG, fg="white",
                    activebackground="#1E293B",
                    relief="flat", bd=2,
                    command=lambda r=row, c=col:
                        self.make_move(r, c)
                )
                button.grid(row=row, column=col,
                            padx=3, pady=3)
                button_row.append(button)
            self.buttons.append(button_row)

        controls = tk.Frame(root, bg=BG)
        controls.pack(pady=10)

        tk.Button(
            controls, text="RESTART GAME",
            font=("Arial", 11, "bold"),
            bg="#22C55E", fg="black",
            command=self.restart
        ).pack(side="left", padx=8)

        tk.Button(
            controls, text="EXIT",
            font=("Arial", 11, "bold"),
            bg="#EF4444", fg="white",
            command=root.destroy
        ).pack(side="left", padx=8)

        self.restart()
```

PROGRAM CODE

```
    def restart(self):
        # Clear the board and begin with Player X
        self.board = [""] * 9
        self.current_player = "X"
        self.game_active = True

        self.status.config(
            text="PLAYER X'S TURN",
            fg="#FACC15"
        )

        for row in range(3):
            for col in range(3):
                self.buttons[row][col].config(
                    text="", state="normal",
                    fg="white", bg=BOARD_BG
                )

    def make_move(self, row, col):
        # Ignore moves after the game has ended
        if not self.game_active:
            return

        index = row * 3 + col

        # A cell can only be selected once
        if self.board[index] != "":
            return

        self.board[index] = self.current_player
        colour = X_COLOR if self.current_player == "X" else O_COLOR

        self.buttons[row][col].config(
            text=self.current_player,
            fg=colour,
            disabledforeground=colour,
            state="disabled"
        )

        if self.has_winner():
            self.game_active = False
            self.status.config(
                text=f"PLAYER {self.current_player} WINS!",
                fg=X_COLOR
            )
            messagebox.showinfo(
                "Game Over",
                f"Player {self.current_player} wins!"
            )
            self.disable_board()
            return
```

PROGRAM CODE

```
    def disable_board(self):
        # Prevent any additional moves after the result
        for row in range(3):
            for col in range(3):
                self.buttons[row][col].config(
                    state="disabled"
                )


# Main Application
if __name__ == "__main__":
    root = tk.Tk()
    game = TicTacToe(root)
    root.mainloop()
```

PROGRAM CODE (CONTINUED)

Save the code sections from the previous pages in the same file, in order. The program starts with the imports and ends with the main application block below.

```
# Main Application
if __name__ == "__main__":
    root = tk.Tk()
    game = TicTacToe(root)
    root.mainloop()
```

HOW TO RUN THE PROGRAM

- Install Python 3 if needed.
- Save the complete code in a file named tic_tac_toe.py.
- Open Command Prompt or a terminal in the file's folder.
- Run: python tic_tac_toe.py
- Click the cells to play. Use RESTART GAME to play again or EXIT to close the window.

OUTPUT

Output of the Tic-Tac-Toe game:

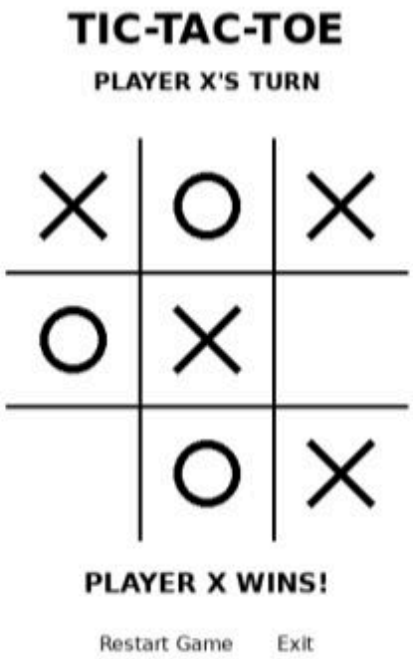


Figure: Sample game output showing Player X winning.

RESULT

Thus, a two-player Tic-Tac-Toe game was developed using Python Tkinter. The game includes a 3×3 board, alternating X and O turns, winner detection, draw detection, result messages, restart functionality, and an exit option. This project demonstrates basic GUI design, event handling, loops, conditions, and game logic.